

Enterprise Data Processing Pipeline with PySpark, SQL and Azure Cloud

Design Document

C-DAC, Bangalore

Project Code – PGCP-BDA-012

July 07, 2026

Prepared By:

Mohd Sohail Siddiqui (Leader)	260250125049
Aryan Shukla	260250125010
Mahendra Kushwaha	260250125044
Divyansh Saxena	260250125021
Chirayu Saraswat	260250125019

Project Code	1
Date	1
Prepared by	1
Introduction	3
Purpose	3
Scope	3
Intended Audience	4
References	4
Acronyms, terms and definitions	4
Assumptions and constraints	5
Basic Design approach	5
Risks	5
System overview	6
Architecture Design	6
Detailed Component Breakdown	7
Data Design	7
Storage Tiering Strategy (Medallion Architecture)	7
Component Design	8
Deep Dive: Module 2 & 3 Process Flow	8
Work-flow Summary w.r.t Identifying Components : -	9
Interface Design	14
Design Test	14
Cross Reference with System Requirement Specification	17
Appendix	18

Introduction

1.1 Purpose

The primary purpose of this comprehensive technical design document is to establish a rigorous blueprint for the development, deployment, and operational maintenance of the Enterprise Data Processing Pipeline. This document translates high-level system requirements into granular architectural representations, structural data models, component logic, and interface designs.

The target system is designed to automate large-scale enterprise data ingestion, handle structured and semi-structured payloads, execute multi-stage clearing and transformation rules, and serve analytical layers with high-performance query execution capabilities. It serves as the definitive reference manual for data engineers, cloud architects, QA professionals, and system administrators throughout the project lifecycle.

1.2 Scope

This document covers the structural implementation boundaries of the Enterprise Data Processing Pipeline engineered at C-DAC Bangalore. The boundaries of this system include:

- Orchestrated data ingestion schedules pulling from transactional environments.
- Schema-conforming raw storage layers and transient data manipulation directories.
- High-performance cloud computing compute environments executing optimized PySpark frameworks.
- Enterprise data warehousing models optimized for analytical star schemas.
- Downstream delivery interfaces comprising semantic reporting layers and Large Language Model (LLM) intelligence generation loops.

Continuous operational tuning, localized micro-benchmarking, and programmatic iterations remain part of the active development cycle.

1.3 Intended Audience

This engineering specification is curated specifically for:

- **Data Engineering Teams:** To directly implement extraction scripts, transformation configurations, and storage schemas.
- **Cloud Solutions Architects:** To validate resource provisioning, performance optimization constraints, and service configurations.
- **Quality Assurance (QA) Engineers:** To design end-to-end unit, integration, and regression test suites.
- **Project Stakeholders & Evaluators:** To verify architectural alignment with foundational design parameters.

1.4 References

1. *Functional Requirement Specifications (FRS) for Enterprise Data Management Systems.*
2. *System Requirement Specification (SRS) - Enterprise Data Processing Pipeline.*
3. *Mid Evaluation Technical Progress Report, C-DAC Bangalore.*
4. *Apache Spark & Azure Databricks Architecture Best Practices Manual.*

2.0 Acronyms, Terms and Definitions

- **ADLS Gen2:** Azure Data Lake Storage Generation 2
- **ADF:** Azure Data Factory
- **ETL:** Extract, Transform, Load
- **ELT:** Extract, Load, Transform
- **RBAC:** Role-Based Access Control
- **KPI:** Key Performance Indicator
- **SRS:** System Requirement Specification

3.0 Assumptions and Constraints

- **Cloud Infrastructure Availability:** The runtime environment depends entirely on the uptime, region allocation, and active tenant status of Microsoft Azure cloud infrastructures.
- **Input Schema Structure:** Source source datasets are assumed to arriving as structured or semi-structured relational data (CSV, JSON, or Parquet) containing explicit index fields that mandate deterministic schema validations.
- **Security Enforcements:** Network topologies must restrict data visibility across processing tiers using strict Azure RBAC profiles, denying unauthorized lateral cross-network inspection.

4.0 Basic Design Approach

The design adopts a highly decoupled, modular micro-batch processing pattern. The primary principle is the separation of compute and storage:

1. **Ingestion Layer:** Purely operational data replication without computational manipulation.
2. **Distributed Compute Tier:** Managed Apache Spark instances processing parallelized datasets across ephemeral memory structures.
3. **Data Warehousing Model:** Highly indexed, column-oriented analytical representations separating analytical tables from staging paths.
4. **Visualization & Insights:** Abstracted layer consuming presentation views without direct structural modification privileges.

5. Risks Assessment and Mitigation Strategies

- **Data Inconsistencies:** High exposure to fluctuating data quality in source schemas.

- **Spark Performance:** Potential processing bottlenecks or high costs without ongoing cluster and notebook resource optimization.
- **Security Breaches:** Mishandling sensitive corporate assets if fine-grained access rules fail to enforce rigorous segregation.

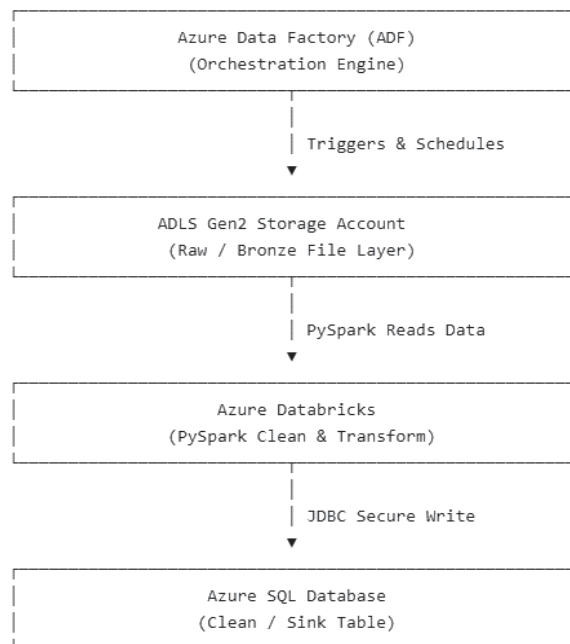
6. System Overview

The Enterprise Data Processing Pipeline addresses the critical challenges of high operational effort, delayed reporting, and poor data quality faced by modern data-driven organizations. By replacing manual processing workflows with an automated cloud-based ecosystem, this architecture ingests large volumes of diverse data, validates structural compliance, applies complex business transformations using PySpark, warehouses information for analytical query speeds, and serves metrics back to business users alongside automated AI insights.

7. Architecture Design

7.1 System Workflow Diagram

The architectural data flow moves systematically across the modern data stack as outlined below:



High-Level End-to-End Technical ETL Pipeline Architecture

7.2 Detailed Component Breakdown

1. Azure Data Factory (ADF)

- **Primary Function:** Acts as the central command center for the entire data pipeline. It does not process the data itself but manages the workflow.
- **Operational Mechanism:** As indicated by the **"Triggers & Schedules"** pathway, ADF is responsible for initiating the pipeline based on predefined chronological schedules (e.g., nightly batch runs) or event-based triggers (e.g., a new file arriving in a storage container).
- **Documentation Context:** In your project, this component guarantees that data movement is fully automated, eliminating the need for manual script execution.

2. ADLS Gen2 Storage Account

- **Primary Function:** Serves as the initial landing zone for raw, unprocessed data extracted from the source systems.
- **Operational Mechanism:** Azure Data Lake Storage Generation 2 (ADLS Gen2) provides a secure, hierarchical file system. The diagram specifies this as the **"Bronze File Layer,"** referencing the Medallion Architecture pattern where data is stored in its exact, original, immutable state before any modifications occur.
- **Data Transition:** The diagram shows the pathway **"PySpark Reads Data"** exiting this component, indicating that the downstream compute cluster mounts this storage account to pull the raw files into memory.

3. Azure Databricks

- **Primary Function:** The heavy-lifting processing tier of the architecture. This is where the actual data manipulation happens.
- **Operational Mechanism:** Utilizing distributed Apache Spark (PySpark), Databricks reads the raw data from the ADLS Gen2 Bronze layer. It applies necessary transformations, which typically include schema validation, handling null values, dropping duplicates, and applying specific business logic.
- **Data Transition:** Once the data is transformed in-memory, the diagram outlines a **"JDBC Secure Write"** operation. This means Databricks establishes an authenticated Java Database Connectivity (JDBC) connection to push the finalized dataset into the target database.

4. SQL Database

- **Primary Function:** The final destination (Sink) for the transformed, structured data.
- **Operational Mechanism:** This component receives the clean data pushed by Databricks. As a relational database, it stores the data in highly structured **"Clean / Sink Tables."**
- **Documentation Context:** By landing the data here, it becomes immediately available for downstream consumption. In a standard enterprise project, this SQL database acts as the serving layer for reporting tools.

8. Data Design & Dimensional Modelling

8.1 Storage Tiering Strategy (Medallion Architecture)

To preserve maximum data integrity throughout validation processes, files follow a strict directory structure:

1. **Raw Ingestion Directory:** Immutable folder where incoming source files are written exactly as received.
2. **Cleaned Processing Directory:** Stores structured Delta Lake tables after validation scripts remove duplicates, fix column types, and drop corrupted rows.
3. **Curated Analytics Directory:** Aggregated corporate summaries organized by operational domains, optimized for fast querying by reporting tools.

9. Component Design

9.1 Core Pipeline Modules

Module 1: Data Ingestion (ADF workflows & raw staging)

Module 2: Data Processing (PySpark validation, handling duplicates)

Module 3: Data Transformation (Business KPIs, structural aggregation)

Module 4: Reports according to the business Development (Business output)

Module 5: AI Insight Generation (LLM automated summaries)

9.2 Deep Dive: Module 2 & 3 Process Flow

The process flow for the PySpark component validation and transformation notebook follows these steps:

1. **Read and Match Schema:** PySpark scripts explicitly mount the ADLS Gen2 path and parse file schemas.
2. **Filter Duplicates:** Record collections discard duplicate keys via active distinct criteria evaluations.

3. **Null Correction:** Fill or handle missing values safely based on predefined business criteria.
4. **Join and Aggregate:** Apply relational operations to match facts with master tables to derive business KPIs.
5. **Export Output:** Write processed data blocks directly into target analytics warehouses.

10. Work-flow Summary w.r.t Identifying Components: -

10.1 Agile Software Development Lifecycle (SDLC) Workflow

The pipeline's development is governed by an Agile Software Development Life Cycle (SDLC) to ensure iterative delivery, continuous testing, and structural modularity.

- **Requirement Analysis:** System requirements and operational bottlenecks (such as high manual effort and delayed reporting) are gathered, analyzed, and mapped into the formal System Requirement Specification (SRS) document.
- **System Design & Architecture:** Cloud infrastructure blueprints are drawn, defining the decoupled Extract, Transform, Load (ETL) principles, storage medians, and computing nodes.
- **Iterative Development:** Modules are programmed sequentially (Ingestion → Processing → Warehousing), utilizing GitHub for version control and strict PEP-8 compliance for Python/PySpark scripts.
- **Quality Assurance & Testing:** Each module boundary undergoes rigorous validation for data quality, schema consistency, and exact business rule execution before integration.
- **Deployment & Maintenance:** The finalized ecosystem is deployed entirely onto Microsoft Azure Cloud, with automated maintenance triggers configured via Azure Data Factory.

10.2. End-to-End Operational ETL Workflow

Once deployed, the live data traverses a strict Medallion-style architecture (Raw → Cleaned → Curated) to preserve data integrity and isolate processing compute workloads.

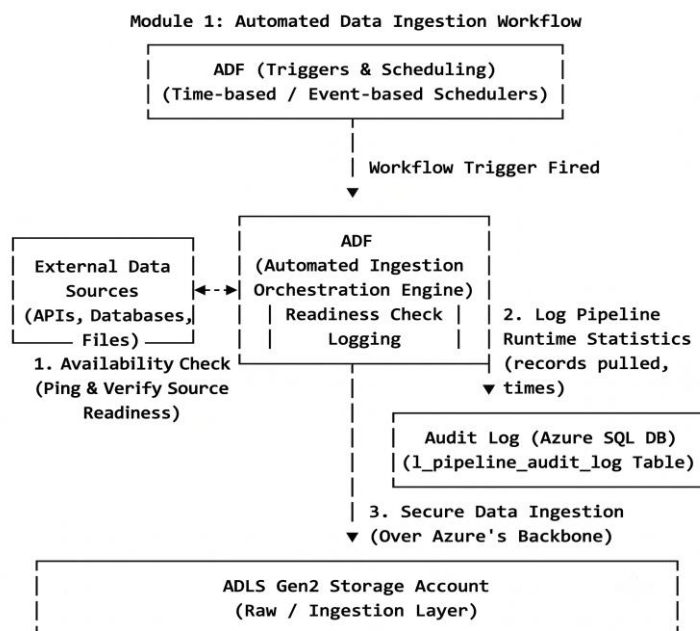
1. **Orchestrated Extraction:** The workflow initiates via automated triggers in Azure Data Factory (ADF) which connect to diverse, fragmented corporate data sources.
2. **Immutable Storage Staging:** ADF streams the unstructured/semi-structured datasets directly into the /raw/ directory of Azure Data Lake Storage Gen2 (ADLS Gen2) without applying any transformations.
3. **Distributed Cluster Invocation:** Upon successful storage load, ADF webhooks automatically wake up the Azure Databricks PySpark clusters to begin the heavy computational tasks.
4. **In-Memory Validation & Cleaning:** Databricks processes the data in distributed memory, strictly validating schemas, removing duplicate Universal Account Numbers (UANs), and imputing null values.
5. **Relational Transformation:** The clean records are joined with master reference tables to calculate high-value Key Performance Indicators (KPIs) and operational metrics.
6. **Enterprise Warehousing:** The fully transformed data arrays are loaded into dedicated SQL pools within SQLdatabase/warehouse Analytics, organized into highly indexed Fact and Dimension tables (Star Schema).
7. **Analytical Delivery:** Power BI connects to the SQLdatabase/warehouse tables to render real-time visual dashboards, while integrated Large Language Models (LLMs) like OpenAI/Gemini scan the metrics to generate automated executive text summaries.

10.3. Detailed Component Execution Workflow (Module by Module)

To clearly document the system's runtime behavior, the pipeline is divided into six distinct operational execution modules.

Module 1: Automated Data Ingestion Workflow

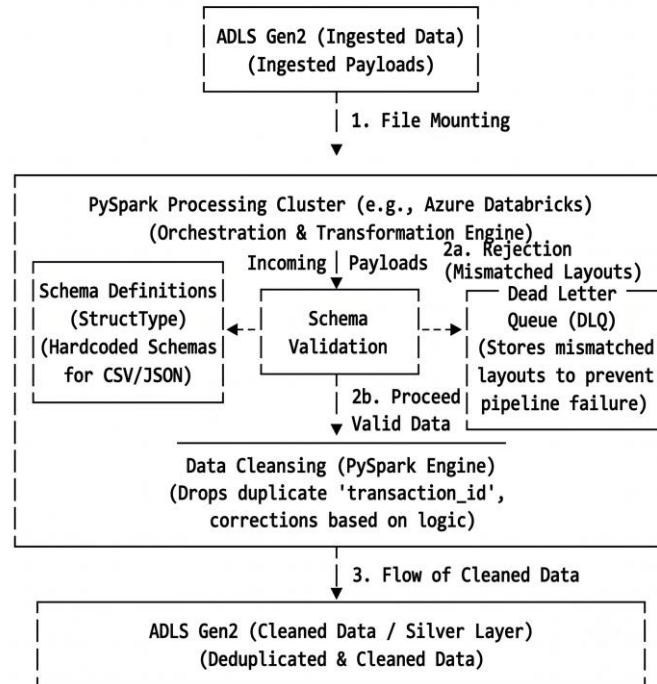
- **Trigger Mechanism:** Time-based or event-based schedulers fire within ADF.
- **Availability Check:** The system pings external databases and APIs to verify source readiness.
- **Ingestion Execution:** Data is replicated securely over Azure's backbone into ADLS Gen2.
- **Logging:** Run statistics (records pulled, execution times) are logged into the `l_pipeline_audit_log` system table.



Module 2: Schema Validation & Processing Workflow

- **File Mounting:** PySpark scripts mount the ADLS Gen2 target paths.
- **Schema Enforcement:** Incoming CSV/JSON payloads are checked against hardcoded StructType schema definitions; mismatched layouts are rejected to a Dead Letter Queue to prevent pipeline failure.
- **Data Cleansing:** The PySpark engine drops duplicate `transaction_id` rows and corrects missing values using pre-defined logical fallbacks.

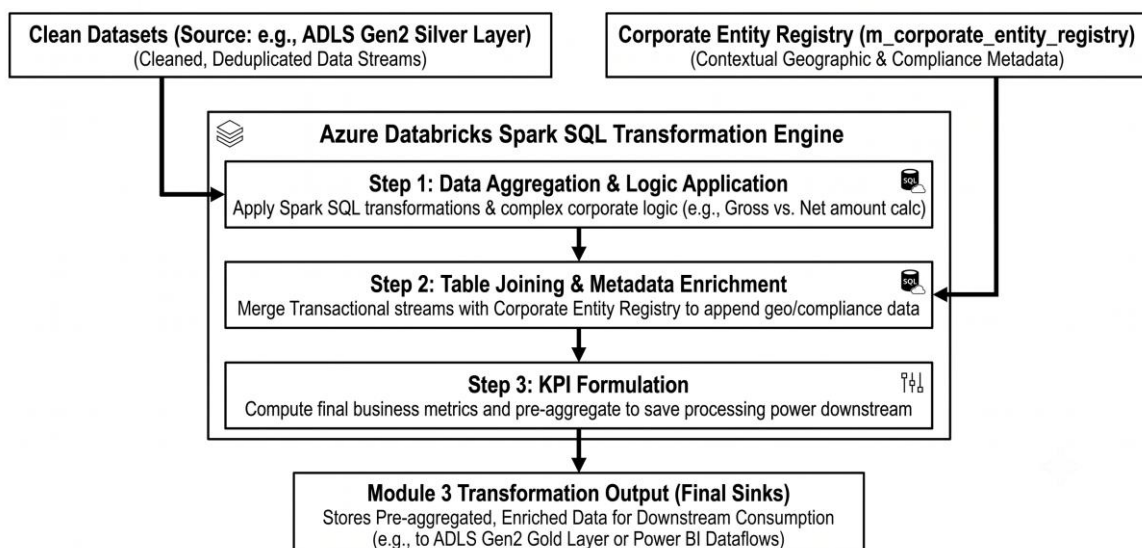
Module 2: Schema Validation & Processing Workflow



Module 3: Business Logic Transformation Workflow

- **Data Aggregation:** The clean datasets undergo Spark SQL transformations where complex corporate logic (e.g., gross vs. net amount calculations) is applied.
- **Table Joining:** Transactional streams are merged with the `m_corporate_entity_registry` to append contextual geographic and compliance metadata.
- **KPI Formulation:** Final business metrics are computed and pre-aggregated to save processing power downstream.

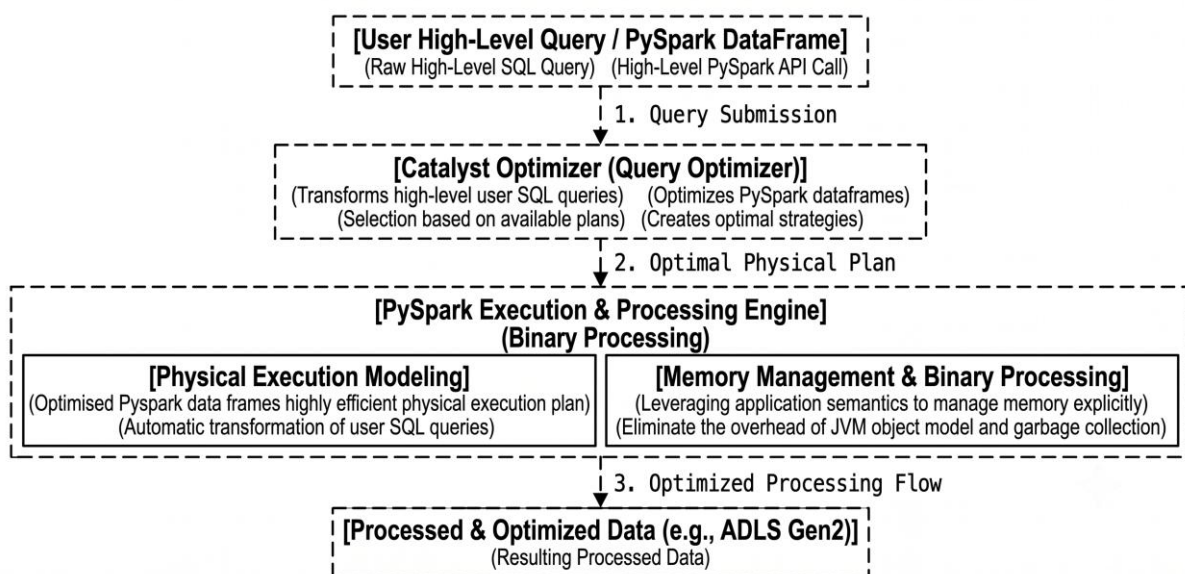
Module 3: Business Logic Transformation Workflow



Module 4: Data Optimization & Processing

- **Physical Execution Modelling:** Optimised Pyspark data frames highly efficient physical execution plan that automatically transforms high level users SQL queries using **Catalyst Optimizer**.
- **Memory Management & Binary Processing:** Leveraging application semantics to manage memory explicitly and eliminate the overhead of JVM object model and garbage collection.

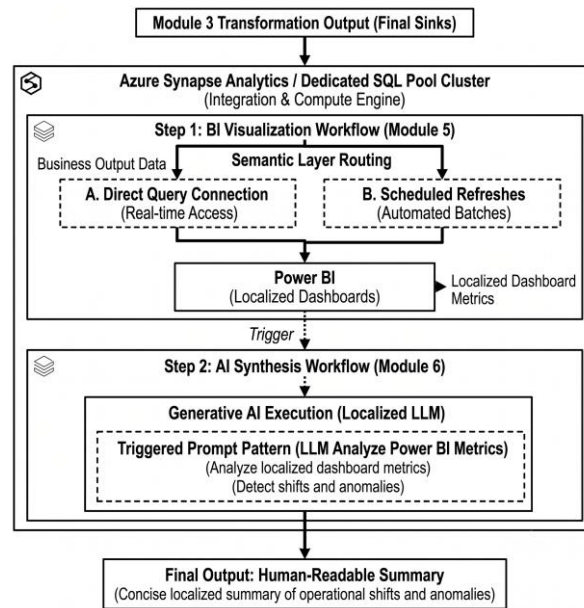
Module 4: Data Optimization & Processing



Module 5 & 6: BI Visualization & AI Synthesis Workflow

- **Semantic Layer Routing:** business output sync with the SQL endpoints using Direct Query or scheduled refreshes.
- **Generative AI Execution:** A localized LLM prompt pattern is triggered, commanding the AI to analyze the localized dashboard metrics and output a concise, human-readable summary of operational shifts and anomalies.

Modules 5 & 6: BI Visualization & AI Synthesis Workflow



11. Interface Design

The target downstream interface is delivered through web components and analytical visualization applications:

- **AI Insight Viewbox:** Text boxes built into reporting tools that display clear, automated enterprise summaries powered by LLM models.
- **Administrative Management:** Interfaces for monitoring job pipelines, inspecting log histories, and handling access controls.

12. Design Test

Note for QA Team: The operational testing verification matrices are to be completed and expanded upon in this section during the integration testing phase.

Proposed Pipeline Validations

- **Data Quality Testing:** Verification that missing or invalid formats are dropped during the validation module phase.
- **Schema Consistency Testing:** Verification that updated source tables do not break downstream Azure Databricks transformations.
- **Business Rules Execution:** Verification that financial aggregation math perfectly mirrors cross-checked internal source DB queries.

1. Data Ingestion & Orchestration Testing (Azure Data Factory)

These tests verify that the orchestration engine successfully pulls data from source systems and lands it immutably in the storage lake.

Test Case ID	Test Category	Test Objective	Test Steps	Expected Result
TC-ING-01	Ingestion	Verify successful scheduled data extraction.	1. Trigger the ADF pipeline via the scheduled event. 2. Monitor the Copy Activity execution logs. 3. Check the target ADLS Gen2 /raw/ directory.	Raw data files are securely replicated to the storage container without any structural modifications.
TC-ING-02	Ingestion	Validate pipeline behavior on source endpoint failure.	1. Temporarily disable access to the source database. 2. Trigger the ADF pipeline. 3. Review the ADF audit logs.	The pipeline catches the timeout error, halts execution, and logs a FAIL status in the runtime statistics.

2. Schema Consistency Testing (Azure Databricks)

These tests ensure that upstream changes in data formats do not crash the downstream distributed processing notebooks.

Test Case ID	Test Category	Test Objective	Test Steps	Expected Result
TC-SCH-01	Schema	Verify ingestion of conforming datasets.	1. Upload a CSV/JSON payload matching the predefined StructType schema. 2. Execute the PySpark read module.	PySpark successfully parses the file and loads it into the data frame memory structure.
TC-SCH-02	Schema	Validate handling of altered source column orders.	1. Submit a source file where column arrangements are randomly shifted. 2. Run the PySpark schema validation module.	The schema validation detects the structural mismatch and routes the corrupted layout to a Dead Letter Queue, pausing downstream tasks

13. Cross Reference with System Requirement Specification

System Requirement Specification Features	Developed/Targeted Components	Adherence / Limitations
REQ-01: Automated Data Ingestion	Azure Data Factory Pipeline Framework	Meets the FR completely: Collects raw data and schedules ingestion jobs directly to ADLS Gen2.
REQ-02: Distributed Processing	Azure Databricks Spark Engine Notebooks	Meets the FR completely: Runs scalable PySpark modules to clean and validate datasets.
REQ-03: Data Warehousing	SQLdatabase/warehouse Data Models	In Progress: Star-schema models are being actively deployed.
REQ-04: AI-Driven Analytics	OpenAI / Gemini Integration Framework	Planned: Automated textual business insights to be integrated next.

Appendix

Appendix A: Technical Environment & Infrastructure Configuration

A.1 Azure Cloud Resource Specifications

A.2 Application & Library Dependencies

Appendix B: Source Data Mockups & Schema Dictionaries

B.1 Sample Raw Data Ingestion Payload (CSV Format)

B.2 Target Enterprise Data Warehouse Schema (SQLdatabase/warehouse)

Appendix C: Extended PySpark Transformation Logic

Appendix D: Pipeline Error Codes & Troubleshooting Matrix

Appendix E: Role-Based Access Control (RBAC) Security Matrix